7 Vector processing

> "If you were plowing a field, which would you rather use: two strong oxen or 1024 chickens?"
>
> *Seymour Cray*

Building on our knowledge of low-level programming, we learn about computing on vectors—we're still computing on single items of data, but these items are composed of several data values. These computations are low-level—in the sense that we're using special CPU registers that can decompose their contents into *lanes* that store separate data values, and special instructions that can compute simultaneously on all lanes!

Notes

- A key idea in vector processing: computing with *scalar* or *packed* values in registers. Packed values encode arrays/vectors of values.
- Registers that store packed values that are organised into a number of *lanes*. For instance two 64-bit values in a single 128-bit register—that is, the 128-bit register is split into two lanes.[1]
- The number of lanes is usually user-configurable. For example, the same 128-bit register from the previous example could be used to store four 32-bit values in that register.

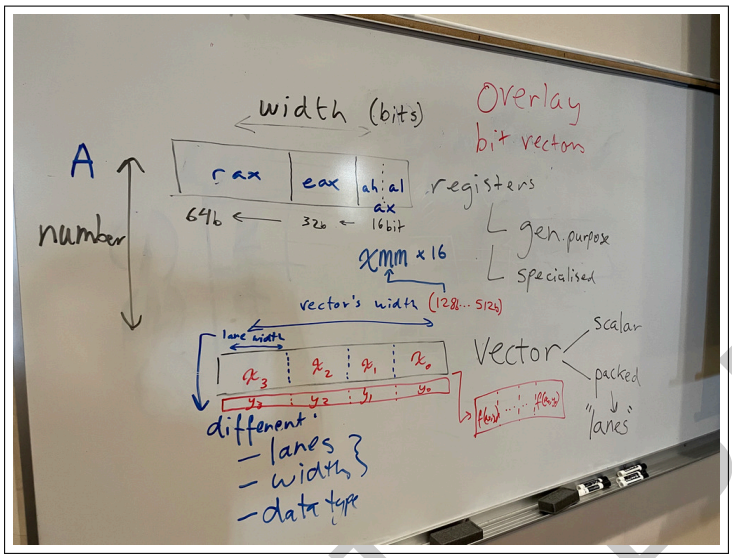
1: The data items in each lane of a vector register are assumed to have the same type—e.g., they are all signed/unsigned integers, or floating-point encodings of a specific precision.

Register Segments vs Lanes

Recall the segmentation of *general-purpose* registers in x86—such as `al`, `ah`, `ax`, `eax` and `rax`. Lanes in *vector* registers are a related by different concept. Instructions that process values in general-purpose register can locate a value in a specific register segment but assume that the register contains that *single* value. Instructions that process values in vector registers are grouped into families (for different numbers of lanes, and the different data that each lane contains) that simultaneously process *multiple* values.

- For our purposes, we can think of "vector registers" as simply storing a bit string. How that register is sub-divided (into lanes) is handled by the *instructions* we use on that register. There are hundreds of vector instructions on modern x86! This is partly because there are many kinds of operations that CPUs support over vectors,[2] but also because there are many different arrangements that CPUs support for those registers.
- Vector extensions to the x86 ISA include: MMX (MM register), SSE (128-bit XMM), various extensions of SSE, AVX (256-bit YMM and later 32 512-bit ZMM from AVX-512). As a standard, any processor

2: For some examples, see PMADDWD, DPPS, PCLMULQDQ, PHMINPOSUW, PCMPxSTRx.



that provides the x86-64 ISA must also support SSE2. MM, XMM, and YMM are segments within ZMM.

- We'll assume having AVX2 support. That gives us 16 256-bit registers (`ymm0`-`ymm15`) and a bunch of operations to operate on various data types (integers and floats) and precisions.[3]
- Compilers can *auto-vectorize* code to use vector processing.
- For more control, you can manually *vectorize* code. This involves specifying the vector registers and instructions to use.
- Rather than hand-vectorizing by using assembly, you can use *intrinsics*.[4] Using intrinsics is usually easier for programmers when compared to writing assembly—even if it's inline assembly, which is arguably a bit easier than starting from an empty file and writing assembly code. There are two reasons for this. First, and most importantly, the compiler is left to figure out register and stack allocations, which is tricky and brittle to do manually. And second, remember that an intrinsic might map to several instructions.
- See the description of vector registers in CS:APP3e [21].
- We saw these examples:

3: MMX initially only supported packed integers.

4: We've already encountered intrinsics on page 30 when we were studying OpenMP.

4→8

Change the parameters of add4floats to be [8] and see what assembly is produced. Is that output what you expected? Did the compiler complain? What if you use "-Wall -Wextra"?

```
1  // gcc example.c -c -S -mavx2
2  #include <immintrin.h>
3
4  void
5  add4floats_plain (float x[4], float y[4], float out[4])
6  {
7    // out[] = x[] + y[]
8    // for each i (where 0 <= i < 3): out[i] = x[i] + y[i]
9    for (int i = 0; i < 4; i++) {
10     out[i] = x[i] + y[i];
11   }
12 }
13
14 void
15 add4floats(float in1[4], float in2[4], float out[4])
16 {
17     __m128 x = _mm_loadu_ps(in1);
18     __m128 y = _mm_loadu_ps(in2);
```

```
    __m128 sum = _mm_add_ps(x, y);
    _mm_storeu_ps(out, sum);
}

void
add8floats(float in1[8], float in2[8], float out[8])
{
    __m256 x = _mm256_loadu_ps(in1);
    __m256 y = _mm256_loadu_ps(in2);
    __m256 sum = _mm256_add_ps(x, y);
    _mm256_storeu_ps(out, sum);
}
```

- Read and simplify the assembly.
- Add -O3 and -mtune=haswell and look at the resulting assembly

Commented instructions

If you use the GNU assembler, remember to use "#" for commenting single-line instructions or C-style delimitations for multi-line comments.

- In code that uses intrinsics, note the following types: `__m256` (single-precision floats), `__m256i` (integers, any precision), `__m256d` (double-precision floats).
- Next we tackle examples that are slightly more complex. Look at the assembly produced by the following:

```
#include "stdio.h"
#include "immintrin.h"
#include "x86intrin.h"

float total;

void
calculate (int NUM_RUNS, int SIZE, float data[])
{
   for (int run = 0; run < NUM_RUNS; run++) {
     total = 0.0f;
     for (int i = 0; i < SIZE; i += 8) {
       total += data[i + 0];
       total += data[i + 1];
       total += data[i + 2];
       total += data[i + 3];
       total += data[i + 4];
       total += data[i + 5];
       total += data[i + 6];
       total += data[i + 7];
     }
   }
}

void
calculate_vec (int SIZE, float data1[], float data2[])
{
   for (int i = 0; i < SIZE; i += 8) {
     data1[i + 0] += data2[i + 0];
     data1[i + 1] += data2[i + 1];
     data1[i + 2] += data2[i + 2];
     data1[i + 3] += data2[i + 3];
     data1[i + 4] += data2[i + 4];
     data1[i + 5] += data2[i + 5];
     data1[i + 6] += data2[i + 6];
     data1[i + 7] += data2[i + 7];
   }
```

```
}

void
calc2 (int SIZE, float data[])
{
  __m256 totalMM;

  total = 0.0f;
  totalMM = _mm256_set_ps(0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0
    f, 0.0f, 0.0f);
  for (int i = 0; i < SIZE; i+=8) {
    __m256 step = _mm256_set_ps(data[i + 0], data[i + 1],
    data[i + 2], data[i + 3], data[i + 4], data[i + 5],
    data[i + 6], data[i + 7]);
    totalMM = _mm256_add_ps(step, totalMM);
  }
  float buf[8];
  _mm256_store_ps(buf, totalMM);
  total = buf[0] + buf[1] + buf[2] + buf[3] + buf[4] + buf
    [5] + buf[6] + buf[7];

}
```

- Compile the above with the following invocations:
  - `$ gcc test3.c -mavx2 -S -o test3.s`
  - `$ gcc test2.c -mavx2 -S -o test2_avx.s`
  - `$ gcc test2.c -S -o test2.s`
  - `$ gcc test1.c -S -o test1.s`
  - `$ gcc test1.c -mavx2 -S -o test1_avx.s`

  We see what happens when we recompile without the "-mavx2" flag.

Optimisation

See what happens in each case when we add "-O" flag, and add higher optimisation levels. What do you think is going on?

- Additional flags to investigate: "-mavx512bw -mtune=haswell -O3 -ftree-vectorize"
- Consider the following code. Once compiled, run it to see it's output. Then inspect the binary using `objdump -S`.

```
// gcc -g -O -mavx2 testvector.c
#include "stdio.h"
#include "immintrin.h"
#include "x86intrin.h"

#define NUM_RUNS 5

#define SIZE (8*64)
float data[SIZE];

int
main () {
unsigned long start_cycles, end_cycles;

for (int i = 0; i < SIZE; i++) {
  data[i] = -1.0f;
}

double total = 0.0f;

for (int run = 0; run < NUM_RUNS; run++) {
  total = 0.0f;
```

```
  start_cycles = __rdtsc();
  for (int i = 0; i < SIZE; i++) {
    total += data[i];
  }
  end_cycles = __rdtsc();
  printf("1: Total=%f Time taken=%lu cycles\n", total,
    end_cycles - start_cycles);
}

for (int run = 0; run < NUM_RUNS; run++) {
  total = 0.0f;
  start_cycles = __rdtsc();
  for (int i = 0; i < SIZE; i+=8) {
    total += data[i + 0];
    total += data[i + 1];
    total += data[i + 2];
    total += data[i + 3];
    total += data[i + 4];
    total += data[i + 5];
    total += data[i + 6];
    total += data[i + 7];
  }
  end_cycles = __rdtsc();
  printf("2: Total=%f Time taken=%lu cycles\n", total,
    end_cycles - start_cycles);
}

__m256 totalMM;

for (int run = 0; run < NUM_RUNS; run++) {
  total = 0.0f;
  totalMM = _mm256_set_ps(0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0
    f, 0.0f, 0.0f);
  start_cycles = __rdtsc();
  for (int i = 0; i < SIZE; i+=8) {
    __m256 step = _mm256_set_ps(data[i + 0], data[i + 1],
    data[i + 2], data[i + 3], data[i + 4], data[i + 5],
    data[i + 6], data[i + 7]);
    totalMM = _mm256_add_ps(step, totalMM);
  }
  end_cycles = __rdtsc();
  float buf[8];
  _mm256_store_ps(buf, totalMM);
  total = buf[0] + buf[1] + buf[2] + buf[3] + buf[4] + buf
    [5] + buf[6] + buf[7];
  printf("3: Total=%f Time taken=%lu cycles\n", total,
    end_cycles - start_cycles);
}

}
```

- Using gdb, "nexti" through the code shown above. Use "info reg all" or "tui reg vector" to see the vector registers. Also remember that you can use "layout asm" and "layout reg", and using a split/src layout if you wish.
- Go back to the SIMD example we had first glimpsed during the analysis described on page 30, and work through the code. (Look for `_ZL6matmulPfS_Ptii._omp_fn.0` in in `yalm/build/asm/src/infer.s`.)